

Sistema E-Ponto

Resumo tecnico do projeto - Flask + REP-P (Portaria MTP n. 671/2021)
UVV - Programacao Avancada para Web

Aplicacao web em Flask para registro eletronico de ponto, projetada para atender as exigencias legais brasileiras de um REP-P (Registro Eletronico de Ponto via Programa). Implementa NSR sequencial, cadeia de hashes SHA-256, geolocalizacao com geofence, exportacao de AFD/AEJ, comprovante em PDF, autenticao 2FA TOTP e RBAC multi-tenant.

1. Stack e estrutura geral

Dependencias principais

- Flask + Flask-SQLAlchemy + Flask-Migrate (banco e migracoes)
- Flask-Login + Flask-Bcrypt + pyotp + qrcode (autenticacao + 2FA TOTP)
- Flask-WTF (CSRF + formularios)
- Flask-Limiter (rate limiting)
- ReportLab (geracao do comprovante e deste proprio PDF)
- psycopg2-binary (suporte a PostgreSQL em producao)

Estrutura de pastas

```
ProjetoE_Ponto/  
  app.py          # factory: create_app()  
  tasks.py        # invoke tasks (install/run/test/seed/zip)  
  pyproject.toml  
  .env.{dev,test,prod}  
  E_Ponto/  
    ext/          # config, db, auth, wtf, limiter, migrate, debugtoolbar  
    models/       # 14 modelos SQLAlchemy  
    views/        # 6 blueprints  
    forms/        # WTForms  
    utils/        # nsr, hashing, geo, clt, afd, aej, pdf, decorators  
    templates/    # Jinja2 + Bootstrap 5
```

A factory **create_app()** inicializa em ordem: config -> db -> register_models -> auth -> migrate -> wtf/limiter (se nao TESTING) -> debugtoolbar -> blueprints.

2. Modelo de dados (14 tabelas)

Tabela	Funcao
users	Usuario (CPF, PIS/NIS, totp_secret, two_factor_enabled)
roles	Papeis: super_admin, admin, rh, funcionario
roles_has_users	M:N com escopo por empresa (unique user+role+business)
businesses	Empresa empregadora (CNPJ + endereco REP-P)
levels	Niveis opcionais anexados ao RoleUser
addresses / cities	Endereco do funcionario

Tabela	Funcao
locais_trabalho	Locais fisicos com lat/lon + raio para geofence
jornadas	Tipos de jornada (padrao, 12x36, 6x1, flexivel)
escalas_funcionarios	Vinculo funcionario - jornada - local com datas
registros	Batidas de ponto (nucleo do sistema)
nsr_sequencias	Sequencial NSR por empresa
retificacoes	Pedidos de correcao (PENDENTE/APROVADA/REJEITADA)
banco_horas	Saldo mensal (positivos, negativos, saldo)
audit_logs	Trilha de auditoria generica (dados_antes/depois)

Destaques da tabela registros

- **nsr** unico por empresa (UniqueConstraint empresa_id+nsr)
- **timestamp_utc**, latitude, longitude, precisao_metros, ip_address
- **suspeito_geo** (boolean - fora do raio do local)
- **hash_registro** e **hash_anterior** formando cadeia SHA-256 (similar a blockchain) - inviolabilidade
- **tipo**: ENTRADA / SAIDA / INCLUSAO / ALTERACAO

3. Os 6 blueprints (camada de rotas)

auth (views/auth.py)

- /auth/login - bcrypt; se 2FA, redireciona para verify-2fa
- /auth/verify-2fa - código TOTP de 6 dígitos
- /auth/setup-2fa - gera secret pyotp + QR Code base64
- /auth/logout

main (views/main.py)

- /index - lista empresas do usuário; salva empresa_id na sessão
- /empresa/<id>/selecionar - troca de empresa ativa

ponto (views/ponto.py) - núcleo funcional

- /ponto/bater - pega último hash -> NSR atômico (FOR UPDATE) -> SHA-256 -> grava com geolocalização + flag suspeito
- /ponto/historico - paginação 30/pag
- /ponto/comprovante/<id> - visualização HTML
- /ponto/comprovante/<id>/pdf - PDF via ReportLab

funcionario (views/funcionario.py)

- /funcionario/ - dashboard pessoal (batidas do dia + retificações)
- /funcionario/retificar/<reg_id> - abre solicitação de correção

rh (views/rh.py) - protegido por role_required(rh, admin, super_admin)

- /rh/ - totais (pendentes, batidas do dia, suspeitos)
- /rh/registros - todas as batidas com filtros por usuário/data
- /rh/retificacoes + /decidir/<id> - aprovar/rejeitar
- /rh/relatorios/afd - gera AFD (Portaria 671/2021)
- /rh/relatorios/aej - gera AEJ (Arquivo Eletrônico de Jornada)

admin (views/admin.py) - role_required(admin, super_admin)

- /admin/ - dashboard com contadores
- /admin/usuarios + /novo - cria usuário com senha temporária
- /admin/locais + /novo - cadastra local com lat/lon/raio
- /admin/jornadas + /nova - cadastra jornada (padrão, 12x36, 6x1, flexível)

4. Utilitarios - inteligencia do REP-P

Arquivo	O que faz
utils/nsr.py	get_next_nsr(empresa_id) - incrementa NSR com with_for_update()
utils/hashing.py	SHA-256 sobre nsr cnpj pis_cpf timestamp tipo hash_anterior + verificar_integridade()
utils/geo.py	Formula de Haversine + verificar_geofence(lat, lon, local)
utils/clt.py	Horas trabalhadas, horas extras, interjornada 11h, tolerancia 5min (CLT art.58)
utils/afd.py	Layout AFD: tipo 1 (header), tipo 3 (batidas), tipo 9 (trailer) em ASCII com pad fixo
utils/aej.py	AEJ mensal por funcionario agrupando marcacoes por dia (ate 6 marcacoes)
utils/pdf.py	Comprovante PDF: empregador, CNPJ, NSR, data/hora, geo, hash
utils/decorators.py	@role_required(*roles) e @empresa_required

5. Seguranca implementada

- **2FA TOTP** (pyotp + qrcode) opcional por usuario
- **Bcrypt** para senhas
- **CSRF** via Flask-WTF (desabilitado em TESTING)
- **Rate limiting** (Flask-Limiter, 200/dia + 50/hora por IP)
- **Cadeia de hashes SHA-256** - alterar um ponto invalida todos os seguintes
- **NSR atomico** com row-level lock (SELECT ... FOR UPDATE)
- **Geofence** com flag suspeito_geo quando fora do raio
- **Auditoria** (audit_logs) com dados_antes/dados_depois
- **RBAC** multi-tenant (papel por empresa via RoleUser)
- **IP do request** registrado em cada batida

6. Ambientes e tasks (invoke)

Comando	Acao
inv install	pip install -e ".[dev,test]"
inv run	carrega .env.dev + flask run (SQLite eponto_dev.db)
inv test	.env.test + pytest
inv prod	.env.prod
inv initdb	db.create_all()
inv seed	cria roles, super_admin (admin@eponto.com / admin123), empresa demo, joao@eponto.com / joao123
inv lint	flake8
inv format	black .
inv zip	empacota excluindo venv/__pycache__/.git

7. Conformidade legal (Portaria MTP 671/2021)

- NSR sequencial por empregador
- Hash de integridade encadeado
- Geracao de AFD e AEJ no layout oficial
- Comprovante imediato em PDF
- Dados do empregador (CNPJ, CEI/CAEPF, endereco)
- Dados do trabalhador (PIS/NIS, CPF)
- Timestamp UTC com timezone
- Tipos de registro (entrada/saida/inclusao/alteracao)
- Retificacao com aprovacao do RH (nao sobrescreve historico)

Resumo em uma frase

E um **REP-P web multi-tenant**: funcionarios batem ponto pelo navegador com geolocalizacao, cada batida vira um Registro imutavel com NSR e hash SHA-256 encadeado, o RH revisa retificacoes e exporta os arquivos AFD/AEJ no formato exigido pela Portaria 671/2021, tudo organizado em blueprints e protegido por login + 2FA TOTP + RBAC por empresa.